University of Palestine

Faculty of software and AI engineering

Department: Artificial Intelligence Engineering

Course: introduction to robotics

Virtual Robot Simulation

Autonomous Navigation Using BFS and Finite State Machine

Student Name: Farah Alwadya

University ID:120220524

Instructor: Fatema alrubaei

March,2026

Content table

1.Introduction 3
2.System Architecture and Design 3
2.1.Overall Structure 3
2.2Environment Design 4
3.Finite State Machine (FSM) Implementation 5
3.1.Robot States 5
3.2.State Transition Logic 6
4.Path Planning Using Breadth-First Search (BFS) 6
4.1.Reason for Choosing BFS 6
4.2.BFS Algorithm Implementation 7
5.Obstacle Avoidance Mechanism 7
6.Movement Logic and Navigation 8
7.Simulation and Visualization (Pygame) 9
8.Testing and Validation 10
9.Challenges Encountered 10
10.Future Improvements 11
11.Conclusion 11
References 12

1. Introduction

Autonomous robot navigation is a fundamental topic in robotics and intelligent systems. A robot must be able to perceive its environment, analyze possible paths, make decisions, and execute movements safely toward a goal.

This project presents a fully implemented 2D Virtual Robot Simulation developed in Python using Pygame. The robot navigates inside a grid-based environment containing obstacles and a goal location. The system was built entirely from scratch without using external AI or pathfinding libraries.

The main objectives of this project are:

- Designing a structured robotic system architecture
- Implementing state-based decision logic
- Applying a shortest-path search algorithm (BFS)
- Developing reactive obstacle avoidance
- Creating a real-time graphical simulation

The project integrates algorithmic thinking with robotics system design principles.

---

2. System Architecture and Design

The system was implemented using a modular architecture to ensure clarity, scalability, and maintainability.

2.1. Overall Structure

The project consists of the following modules:

- Environment class
- Robot class
- Simulation controller

- Visualization module (Pygame)

Environment ↔ Robot ↔ Simulation Loop ↔ Renderer

2.2. Environment Design

The environment is represented as a 2D grid. Each cell in the grid may represent:

- Empty space
- Obstacle
- Goal location

The Environment class handles:

- Boundary checking
- Obstacle detection
- Goal verification

```
def is_within_bounds(self, x, y):
    # check if the robot still in bound and return true
    return 0 <= x < self.width and 0 <= y < self.height
```

This function ensures that the robot never moves outside the grid limits.

2.3 Robot Model

The robot is represented by:

- Current position (x, y)
- Current direction (NORTH, EAST, SOUTH, WEST)
- Internal state (**FSM state**)

The robot interacts with the environment through:

- Sensing neighboring cells
- Calculating path

- Updating internal state
- Executing movement

This separation between environment logic and robot logic ensures clean system design.

---

3. Finite State Machine (FSM) Implementation

To manage robot behavior systematically, a Finite State Machine (FSM) was implemented.

3.1. Robot States

The robot transitions between the following states:

- IDLE
- MOVING
- AVOIDING
- FINISHED

Each state has a specific behavioral responsibility.

```
class RobotState(Enum):
    IDLE = 0  # not moving
    MOVING = 1
    AVOIDING = 2  # avoid obstacles and walls
    FINISHED = 3
```

Figure 2: Robot Finite State Machine

3.2. State Transition Logic

The state transitions are triggered based on environmental feedback and algorithm outcomes.

```
# IDLE -> start moving #if robot doesn't move t
if self.state == RobotState.IDLE:
    self.state = RobotState.MOVING
    if front_sensor == "CLEAR":
        self.move_forward(environment)
    return

# MOVING -> follow BFS #change status if there
if self.state == RobotState.MOVING:
    if front_sensor in ["OBSTACLE", "WALL"]:
        self.state = RobotState.AVOIDING
        return
```

This logic ensures that when movement fails due to an obstacle, the robot switches to reactive avoidance behavior.

The FSM guarantees structured and predictable system behavior.

---

4. Path Planning Using Breadth-First Search (BFS)

Efficient navigation requires calculating the shortest possible path to the goal. For this purpose, the Breadth-First Search (BFS) algorithm was implemented.

4.1. Reason for Choosing BFS

BFS is suitable for this environment because:

- The grid is unweighted
- It guarantees shortest path
- It is deterministic
- It is simple and reliable

4.2. BFS Algorithm Implementation

The algorithm works as follows:

1. Initialize a queue with the starting position
2. Explore all valid neighboring cells
3. Avoid revisiting explored cells
4. Stop when goal is found
5. Reconstruct shortest path

```
def find_path_bfs(self, environment, goal_x, goal_y):
    visited = set()  # set to save the visited cells
    queue = deque()  # used for bfs to allow o(1)pop and append from both tails
    queue.append((self.x, self.y, []))  # add the intiail position to the queque
    visited.add((self.x, self.y))  # add the intiail position to the visited set

    while queue:
        cx, cy, path = queue.popleft()  # cx,cy position of the current cell
        if (cx, cy) == (goal_x, goal_y):
            return path
est (-1 within x axsis, 0 at y)east(+1 at x axsis, 0 at y)
outh(+1 at y axsis)north(-1 at y and 0 at x)
        for dx, dy, move_dir in [(-1, 0, Direction.WEST), (1, 0, Direction.EAST),  #
                                 (0, -1, Direction.NORTH), (0, 1, Direction.SOUTH)]:
            nx, ny = cx + dx, cy + dy  # next cell coordinates
            if environment.is_within_bounds(nx, ny) and not environment.is_obstacle(
                    nx, ny) and (nx, ny) not in visited:
                visited.add((nx, ny))  # if the cell can be visited add it to path
                queue.append((nx, ny, path + [move_dir]))  # update the new path
    return []
```

Figure 3: Shortest Path Generated by BFS

---

5. Obstacle Avoidance Mechanism

In addition to global path planning, a reactive obstacle avoidance system was implemented.

If an unexpected obstacle blocks the path:

- Robot switches to AVOIDING state
- Scans possible directions
- Prioritizes safe directions

- Attempts alternative movement
- This hybrid strategy combines:
- Deliberative planning (BFS)
- Reactive decision-making (real-time avoidance)

```
# AVOIDING -> reactive
if self.state == RobotState.AVOIDING:
    # now we need to check all direction and to use 360 sensor
    sensors = self.sense_all_directions(environment)
    priority_dirs = [  # decide direction to try with first
        Direction((self.direction.value + 1) % 4),  # right
        Direction((self.direction.value - 1) % 4),  # left
        self.direction,                             # front
        Direction((self.direction.value + 2) % 4)   # back
    ]
    moved = False
    for d in priority_dirs:
        if sensors[d.name] in [
                "CLEAR", "GOAL"]:  # choose a clear direction that having the goal
            # turn towards that direction
            while self.direction != d:  # if we still didn't reach the goal
                diff = (d.value - self.direction.value) % 4
                if diff == 1:
                    self.turn_right()
                elif diff == 2:
                    self.turn_left()
                    self.turn_left()
```

Such hybrid systems are commonly used in real robotics applications.

---

6. Movement Logic and Navigation

The robot moves step-by-step according to calculated path or avoidance logic.

```
def move_forward(self, environment):
    nx, ny = self.next_position()
    if not environment.is_within_bounds(nx, ny) or environment.is_obstacle(
            nx, ny):  # if out of bounds, or there an obstacles don't move
        return False
    self.x, self.y = nx, ny  # else, update position
    return True
```

This function ensures safe movement and prevents illegal transitions.

The robot updates its direction before executing movement to ensure correct orientation.

---

7. Simulation and Visualization (Pygame)

The graphical simulation was implemented using Pygame.

The visual interface displays:

- Grid layout
- Robot position
- Obstacles
- Goal cell

```
def draw_grid(self):
    for y in range(self.environment.height):  # loop
        for x in range(self.environment.width):  # lo
            rect = pygame.Rect(
                x * self.cell_size,
                y * self.cell_size,
                self.cell_size,
                self.cell_size

            )

            cell = self.environment.grid[y][x]

            if cell == CellType.EMPTY:
                color = (255, 255, 255)  # white cell
            elif cell == CellType.OBSTACLE:
                color = (0, 0, 0)  # black cell
            elif cell == CellType.GOAL:
                color = (0, 255, 0)  # green cell
```

The simulation loop continuously:

- Updates robot state

- Redraws environment
- Handles execution timing

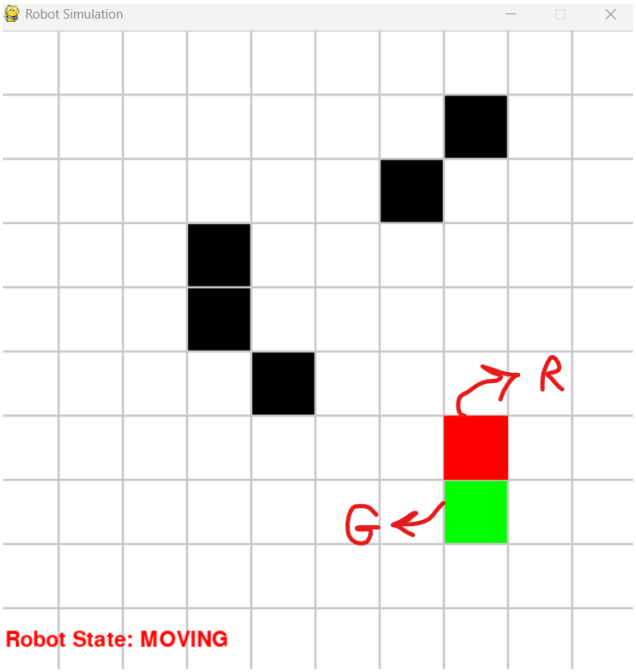

Figure 4: Real-Time Simulation Interface

---

8. Testing and Validation

Multiple test cases were performed to validate correctness:

- Robot starting near boundary
- Robot surrounded by obstacles
- Goal placed in corner
- Complex obstacle configurations

The robot successfully navigated all valid configurations where a path existed.

Edge cases such as blocked paths were handled without system crash.

---

9. Challenges Encountered

During development, the following challenges were addressed:

- Designing a clean and functional FSM
- Debugging BFS path reconstruction as the robot continue falling into infinite loops
- Handling corner and boundary cases
- Synchronizing movement logic with visualization
- Ensuring no infinite loops occur

These challenges strengthened understanding of robotics logic systems.

---

10. Future Improvements

Possible enhancements include:

- Implementing A* algorithm
- Adding weighted grids
- Introducing moving obstacles
- Implementing multi-robot coordination
- Adding sensor noise simulation

These improvements would increase realism and system complexity.

---

11. Conclusion

This project demonstrates a complete autonomous navigation system built from scratch. By combining Finite State Machine logic with BFS path planning and reactive obstacle avoidance, the robot successfully navigates complex grid environments.

The modular design, structured logic, and real-time simulation collectively reflect strong understanding of robotics programming and algorithm implementation.

The system is reliable, extensible, and serves as a solid foundation for advanced intelligent robotics systems.

12. References

- Python Official Documentation : https://docs.python.org/3/

- Pygame Documentation :https://www.pygame.org/docs/
- "Breadth-first search": Wikipedia article — https://en.wikipedia.org/wiki/Breadth-first_search

- Finite State Machine in English ,concept, ex and apps(YouTube): https://www.youtube.com/watch?v=mvpjHBy8t4A
Used to understand FSM concept and how robot changes states.

- Finite State Machine in Python (YouTube): https://python-course.eu/applications-python/finite-state-machine.php
Code explanation of FSM implementation in Python.

- Pygame documentation: https://www.pygame.org/news